A MINI-PROJECT REPORT ON
SUPERMARKET BILLING SYSTEM

SUBMITTED BY

AYUSH JAIN (ROLL NO: SCB42)
SANIKA SALOKHE (ROLL NO: SCB52)
SHREYAS AHIVALE (ROLL NO: SCB57)

CLASS: S Y B. TECH YEAR ENGINEERING

SUBJECT: OBJECT-ORIENTED PROGRAMMING



DEPARTMENT OF COMPUTER ENGINEERING

INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY
An Autonomous Institute affiliated to Savitribai Phule Pune University

HINJEWADI, PUNE, MAHARASHTRA 411057

AY: 2026 -2027 SEM-I

MINI PROJECT DETAILS AND SDGs/ POs/ PSOs MAPPING

Table I: SDGs MAPPING

Roll No	Full Name	Mini Project Title	Domain	SDG No	SDG Justification
SCB42 SCB52 SCB57	AYUSH JAIN SANIKA SALOKHE SHREYASH AHIVALE	Supermarket Billing System	Retail information systems	SDG 9	it demonstrates a digital system for basic retail billing and inventory management.
SCB42 SCB52 SCB57	AYUSH JAIN SANIKA SALOKHE SHREYASH AHIVALE	Supermarket Billing System	Billing and inventory management	SDG 12	quantity and stock tracking can support more informed inventory use.

Table II: POs/PSOs MAPPING

Mini Project Title	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
Supermarket Billing System	3	2	2	0	2	0	0	0	2	0	1

PO1 - 3: Applied computing and C++ concepts, including classes, inheritance, runtime polymorphism, input validation, file handling, and billing calculations.

PO2 - 2: Identified basic retail needs—product management, cart operations, stock checking, and billing—and translated them into application requirements.

PO3 - 2: Designed a working solution using classes such as Item, Cart, Inventory, Customer, Admin, and Bill, with CSV-based persistence and separate customer/admin workflows.

PO5 - 2: Used C++ and standard-library tools such as console streams, file streams, vectors, and CSV processing to implement the system.

PO9 - 2: The menus, prompts, validation messages, receipts, and project documentation communicate system information to users and readers.

PO11 - 1: Applying concepts such as inheritance and runtime polymorphism demonstrates learning and adaptation, though the project does not directly demonstrate ongoing learning or adapting to emerging technologies.

TABLE OF CONTENTS

TITLE	PAGE NO.
1. INTRODUCTION	1
1.1 Motivation	1
1.2 Problem Definition	1
2. METHODOLOGY	2
2.1 Classes and Members	2
2.2 Algorithms	7
2.3 Software and Hardware Details	8
2.4 Libraries/Packages Used	9
3. SOURCE CODE	10
3.1 Customer Panel	11
3.2 Admin Panel	12
4. RESULTS/OUTPUT	14
4.1 Dashboard Selection	14
4.2 Customer Menu	14
4.3 Admin Menu	15
4.4 Checkout/Bill	15
5. CONCLUSION	16
6. REFERENCES	17

1. INTRODUCTION

The Supermarket Billing and Inventory Management System is a menu-driven C++ console application for basic retail checkout. It serves customers who need to browse stock, build a cart, and receive a calculated receipt, and an administrator who maintains product records. The system stores inventory in inventory.csv and appends completed transaction details to transactions.csv, allowing data to remain available between program runs. The implementation demonstrates object-oriented programming through Item, CartItem, Cart, Inventory, Customer, Admin, and Bill classes; it also uses functions, vectors, input validation, file streams, and CSV parsing. The supplied project contains main.cpp, inventory.h, customer.h, bill.h, inventory.csv, and transactions.csv.

1.1 Motivation

Small stores often need a simple way to maintain product quantities and calculate a checkout total consistently. Manual calculations and unstructured records can lead to arithmetic mistakes, overselling, or difficulty reviewing previous sales. This project was selected as a practical application of object-oriented programming: familiar store activities can be represented as software entities with clear data and responsibilities. A lightweight console interface and CSV files keep the implementation understandable and runnable without a database server or third-party framework.

1.2 Problem Definition

The problem addressed is the coordination of a small product catalogue, customer cart, stock availability, and itemized bill in a single application. The intended users are a shopper and a store administrator. The administrator should be able to add, remove, view, and modify products; a customer should be able to view inventory, add or remove products from a cart, change quantities, and check out. Each item carries an ID, name, price, stock quantity, tax percentage, and discount percentage. The expected outcome is a validated checkout total, a receipt, a persistent transaction record, and reduced inventory for purchased quantities. The current application does not process real payments or manage multiple concurrent users.

2. METHODOLOGY

The project follows a requirements-to-implementation methodology. Store products, shopping carts, customer details, administrator access, and bills are represented by C++ classes. The program entry point coordinates the console menus and the two user workflows. Product and transaction data are persisted in CSV files, while a customer's cart exists in memory during the shopping session.

The system is organized around these activities:

- Initialize the inventory file and determine the next bill number.
- Accept a customer or administrator workflow.
- Read, validate, and process user input.
- Load or update product records through the inventory component.
- Calculate and persist checkout transactions.
- Display the receipt or report the operation result.

2.1 Classes and Members

The classes encapsulate product, cart, customer, administrator, inventory, and billing responsibilities. CartItem inherits from Item to reuse product data and specialize item display with a quantity. Item::displayItem() is virtual, and CartItem::displayItem() overrides it. CartItem::displayCartItem() calls the display method through an Item reference, so C++ selects the override at runtime. The Cart and Bill classes contain vectors of cart items.

2.1.1 Item Class

Item represents one inventory product.

Private Members:

- string id : Product id
- string name : Product display name
- double price : Unit price
- int stock : Available quantity

- double tax : Tax percentage
- double discount : Discount Percentage

Methods:

- itemDetails() displays all product fields.
- Virtual displayItem() displays a compact inventory row. A CartItem override adds the cart quantity when called polymorphically.
- getId(), getName(), getPrice(), getStock(), getTax(), and getDiscount() return individual values.
- reduceStock(quantity) rejects a non-positive quantity or a quantity greater than the current stock; otherwise, it deducts the quantity.
- modifyItem(field, newValue) updates the name (n), price (p), stock (s), tax (t), or discount (d). Numeric changes are parsed and validated; discount must be between 0 and 100 percent.

2.1.2 CartItem Class

CartItem represents one product selection in a customer's cart and publicly derives from Item. The selected product initializes the Item base subobject; it is not stored as a separate Item member.

Private Members:

- int quantity : Number of units selected.

Methods:

- The constructor receives an Item and a quantity, initializing its Item base with the product.
- displayItem() overrides the virtual base method. It displays the inherited product details and then the cart quantity.

- displayCartItem() calls displayItem() through an Item reference. This is the runtime-polymorphism example: although the reference has base type Item, the actual object is a CartItem, so the derived override runs.
- setQuantity(newQuantity) changes the selected quantity.
- getQuantity() returns the selected quantity. Product getters such as getId(), getStock(), and getPrice() are inherited from Item.

2.1.3 Cart Class

Cart manages the current customer's in-memory selection.

Private Members:

- vector<cartItem> cartList : Products and quantities selected during this customer session

Methods:

- addItem(item) rejects invalid or over-stock quantities. If the product is already present, it adds to the existing quantity only when the combined quantity remains within the product's stored stock.
- removeItem(itemId) removes the matching product and reports whether it was found.
- modifyCartItem(itemId, field) accepts q to prompt for a new quantity or r to remove the product.
- getCartList() returns a copy of the cart entries for display or checkout.
- clear() empties the cart after a successful checkout.

2.1.4 Inventory Class

Inventory provides file-backed product catalogue operations.

Private Members:

- string fileName : Path/name of the inventory CSV file

Methods:

- createFileIfNeeded() creates the inventory file with its header if the file does not exist.
- findItem(itemId, result) scans valid product rows and copies a matching product into result.
- displayItems() reads and displays valid product rows.
- addItem(newItem) checks for a duplicate product ID and appends a valid product row.
- reduceStock(itemId, quantity) loads a product, reduces its stock, and persists the changed record.
- removeItem(itemId) removes a product row from the file.
- displayItemDetails(itemId) displays one product or reports that it was not found.
- modifyItem(itemId) prompts for a field and value, validates the change, and persists it.
- replaceItem(itemId, replacement, removeRow) rewrites the inventory through inventory.tmp, replacing or omitting the selected record.

2.1.5 Customer Class

Customer stores details for the current customer.

Private Members:

- string name : Customer name
- string phoneNumber : Customer phone number

2.1.6 Admin Class

Admin holds the configured administrator credentials.

Private Members:

- string username : configured administrator username.
- string password : Configured administrator password.

2.1.7 Bill class

Bill represents one checkout.

Private Members:

- string billID : Receipt identifier, such as BILL1.
- string customerName : Customer name for the receipt and transaction rows.
- string customerPhoneNumber : Customer phone number for the receipt and transaction rows.
- vector<cartItem> cartList : Items included in this bill.
- double totalTax : Sum of calculated Tax.
- double totalDiscount : Sum of calculated discounts.
- double total : Sum of line subtotals.
- double finalTotal : Amount due after tax and discounts.

Methods

- The constructor receives the bill ID, customer name, customer phone number, and cart entries.
- calculateBill() calculates item subtotals, taxes, discounts, and the final total.
- displayBill() prints the receipt, including customer contact information.
- saveToCsv(fileName) appends one transactionrow per cart item to the transaction file. It also calls the private ensureTransactionSchema() helper to support the older transaction CSV format.
- ensureTransactionSchema(fileName) recognizes the previous 12-column transaction header and migrates valid historical rows by adding an empty final phone-number column. An unknown header or malformed legacy row stops the save instead of silently rewriting it.

2.2 Algorithms

2.2.1 Algorithm: Startup and bill-number initialization

Input: Inventory and transaction CSV files.

Output: Ready-to-use inventory and the next bill number.

- Open inventory.csv.
- If it does not exist, create it and write the inventory header.
- Open transactions.csv.
- If it cannot be opened, use bill number 1.
- Otherwise, skip the header and parse each transaction row.
- For each valid bill ID beginning with BILL, parse its numeric suffix.
- Track the highest valid suffix.
- Return the highest suffix plus 1; if no valid bill ID was found, return 1.

2.2.2 Algorithm: Add an item to the cart

Input: Product ID and current cart.

Output: Updated cart or an error.

- Read the product ID.
- Call Inventory::findItem() to locate the product.
- If no valid product is found, report that the item was not found.
- Create a CartItem with the found Item and quantity 1.
- In Cart::addItem(), reject a non-positive quantity or one greater than the item's stored stock.
- If the product already exists in the cart, calculate the combined quantity.
- Reject the addition if the combined quantity exceeds the stored stock; otherwise update the existing cart quantity.
- If the product is not already in the cart, add the new CartItem.

2.2.3 Algorithm: Customer checkout

Input: Customer name and phone, current cart, inventory CSV, and transaction CSV.

Output: Receipt, transaction rows, and reduced stock when successful.

- Get a copy of the current cart using Cart::getCartList().
- If the cart is empty, report that it is empty and stop checkout.
- For each cart entry, look up the product again in the inventory file.
- If any product is missing or its current stock is less than the requested quantity, report the checkout failure and stop.
- Construct a Bill with the next bill ID, customer name, customer phone number, and cart entries.
- Calculate each item's subtotal, tax, and discount:

  item subtotal = unit price * quantity

  item tax = item subtotal * tax percentage / 100

  item discount = item subtotal * discount percentage / 100
- Sum line subtotals, taxes, and discounts:

  total amount due = subtotal total + tax total - discount total
- Save the bill rows to transactions.csv. If saving fails, stop before reducing stock.
- For each cart entry, call Inventory::reduceStock() with its item ID and quantity.
- If any stock update fails, report an error. Previously written transaction or inventory changes are not rolled back.
- If all stock updates succeed, increment the bill number, display the receipt, clear the cart, and report completion.

2.3 Software and Hardware Details

- **Programming language:** C++11 or newer.
- **Compiler:** A C++ compiler that supports C++11 or newer.
- **User interface:** A text-based console application.
- **Libraries:** C++ standard library only; no third-party packages or frameworks are required.

- **Data storage:** Local CSV files—inventory.csv for products and transactions.csv for completed bill lines.
- **Operating system:** Windows or Linux with terminal ANSI support

2.4 Libraries/Packages Used

No third-party packages, frameworks, or external APIs are used. The implementation relies on the C++ standard library:

- **iostream** for console input/output.
- **fstream** for CSV persistence.
- **iomanip** for aligned tables and two-decimal currency output.
- **cmath** for finite-number checks.
- **cstdlib** and **climits** for conversion and integer bounds.
- **string** and **vector** for text and collections.
- **cctype** for case-insensitive field selection.
- **cstdio** for file removal/renaming.

These headers are included in inventory.h, while the other project headers reuse that common implementation. The project has no separate package manifest or automated dependency installation step.

3. SOURCE CODE

Program startup (main.cpp)

```
Inventory inventory("inventory.csv");
if (!inventory.createFileIfNeeded())
    return 1;
int nextBillNumber = getNextBillNumber("transactions.csv");
```

Customer checkout (main.cpp)

```
bool inStock = true;
for (auto &cartItem : cartList)
{
    Item item;
    if (!inventory.findItem(cartItem.getId(), item) ||
        item.getStock() < cartItem.getQuantity())
    {
        inStock = false;
        break;
    }
}
if (!inStock)
{
    error("Checkout failed: an item is no longer available in the requested quantity.");
    break;
}
Bill bill("BILL" + to_string(nextBillNumber), customer.getName(),
          customer.getPhoneNumber(), cartList);
bill.calculateBill();
if (!bill.saveToCsv("transactions.csv"))
    break;
bool stockUpdated = true;
for (auto &cartItem : cartList)
{
    if (!inventory.reduceStock(cartItem.getId(), cartItem.getQuantity()))
    {
        stockUpdated = false;
        break;
    }
}
if (!stockUpdated)
{
    error("Could not update inventory stock. Please check the inventory file.");
    break;
}
++nextBillNumber;
bill.displayBill();
cart.clear();
```

Bill calculation (bill.h)

```
for (auto &item : cartList)
{
    double itemTotal = item.getPrice() * item.getQuantity();
    total += itemTotal;
    totalTax += itemTotal * (item.getTax() / 100);
    totalDiscount += itemTotal * (item.getDiscount() / 100);
}
finalTotal = total + totalTax - totalDiscount;
```

Other source files

customer.h collects customer details and checks administrator login; inventory.h implements item, cart, and inventory operations with input validation and CSV storage; bill.h formats receipts and writes transaction records.

3.1 Customer Panel

The customer workflow begins when Customer is selected from the welcome menu. The program collects the customer name and phone number, creates an empty cart, and displays the customer menu.

Customer details (customer.h)

```
title("CUSTOMER DETAILS");
cin.ignore();
name = readLine("  Name: ");
phoneNumber = readLine("  Phone number: ");
```

Customer menu (main.cpp)

```
showMenu("CUSTOMER MENU", {
    "View inventory",
    "Add item to cart",
    "Remove item from cart",
    "Modify item in cart",
    "View cart",
    "Checkout",
    "Return to user selection"
});
```

The customer can view inventory, add an item by ID, remove an item, change its quantity or remove it from the cart, and view the current cart. The program checks that an item exists and that the requested quantity does not exceed available stock.

Checkout and receipt (main.cpp)

```
if (cartList.empty())
{
    info("Cart is empty.");
    break;
}
...
Bill bill("BILL" + to_string(nextBillNumber), customer.getName(),
          customer.getPhoneNumber(), cartList);
bill.calculateBill();
if (!bill.saveToCsv("transactions.csv"))
    break;
...
bill.displayBill();
cart.clear();
```

Before saving the bill, checkout checks stock again. On a successful transaction, the system reduces inventory stock, displays the receipt, and clears the cart.

3.2 Admin Panel

The administrator workflow is handled by runProgram() in main.cpp. The Admin object checks the entered username and password. If login fails, the program displays an error and returns to user selection; after a successful login, it opens the admin menu.

Administrator login (customer.h and main.cpp)

```
bool login(string inputUsername, string inputPassword)
{
    return inputUsername == username && inputPassword == password;
}
...
username = readValue<string>("  Username: ");
password = readValue<string>("  Password: ");
if (!admin.login(username, password))
{
    error("Invalid username or password.");
    continue;
}
```

The menu provides six choices: add an item, remove an item, view inventory, display item details, modify an item, or return to user selection. It repeats until the administrator chooses to return.

Admin menu (main.cpp)

```
showMenu("ADMIN MENU", {
    "Add item to inventory",
    "Remove item from inventory",
    "View inventory",
    "Display item details",
    "Modify item",
    "Return to user selection"
});
```

- Choice 1 validates item values and calls Inventory::addItem(), which rejects duplicate IDs.
- Choice 2 calls Inventory::removeItem().
- Choice 3 calls Inventory::displayItems()
- Choice 4 calls Inventory::displayItemDetails()
- Choice 5 calls Inventory::modifyItem(). During modification, Item::modifyItem() validates the selected field and value before the change is saved.

4. RESULTS/OUTPUT

4.1 Dashboard Selection (Start Page)

```
=== WELCOME TO THE STORE ===
[1] Customer
[2] Admin
[3] Exit
Select an option:
```

Figure 1

4.2 Customer Menu

```
=== CUSTOMER MENU ===
[1] View inventory
[2] Add item to cart
[3] Remove item from cart
[4] Modify item in cart
[5] View cart
[6] Checkout
[7] Return to user selection
Select an option: 1

=== STORE INVENTORY ===
ITEM ID     ITEM NAME                 PRICE    DISCOUNT
-----------------------------------------------------------
1001        Apples               Rs    80.00        5.00%
1002        Grapes               Rs    50.00        0.00%
1003        Milk                 Rs    60.00       10.00%
1004        Bread                Rs    45.00        0.00%
1005        Biscuits             Rs    30.00        0.00%
1006        Rice 5kg             Rs   320.00        5.00%
1007        Tea 250g             Rs   140.00       10.00%
1008        Cooking Oil 1L       Rs   160.00        5.00%
1100        Basmati Rice 250g    Rs    54.00        0.00%
```

Figure 2

```
=== CUSTOMER MENU ===
[1] View inventory
[2] Add item to cart
[3] Remove item from cart
[4] Modify item in cart
[5] View cart
[6] Checkout
[7] Return to user selection
Select an option:

=== YOUR CART ===
ITEM ID     ITEM NAME                 PRICE    DISCOUNT    QUANTITY
-------------------------------------------------------------------
1001        Apples               Rs    80.00        5.00%          3
1002        Grapes               Rs    50.00        0.00%          2
1003        Milk                 Rs    60.00       10.00%          1
1004        Bread                Rs    45.00        0.00%          1
```

Figure 3

4.3 Admin Menu

```
=== WELCOME TO THE STORE ===
[1] Customer
[2] Admin
[3] Exit
Select an option: 2

=== ADMIN SIGN IN ===
  Username: storeAdmin
  Password: admin123
Success: Signed in as administrator.

=== ADMIN MENU ===
[1] Add item to inventory
[2] Remove item from inventory
[3] View inventory
[4] Display item details
[5] Modify item
[6] Return to user selection
Select an option: 1
  Item ID: 1099
  Item name: Pineapple
  Price: 80
  Stock: 30
  Tax percentage: 5
  Discount percentage: 50
Success: Item added to inventory.
```

Figure 4

4.4 Checkout/Bill

```
=== CUSTOMER MENU ===
[1] View inventory
[2] Add item to cart
[3] Remove item from cart
[4] Modify item in cart
[5] View cart
[6] Checkout
[7] Return to user selection
Select an option: 6

=== CHECKOUT RECEIPT ===
Receipt: BILL1017
Customer: Ayush
Phone: 0000110210
------------------------------------------------------------
ITEM ID     ITEM NAME                 PRICE    DISCOUNT    QUANTITY
------------------------------------------------------------------------
1001        Apples              Rs    80.00        5.00%           3
1003        Milk                Rs    60.00       10.00%           2
1002        Grapes              Rs    50.00        0.00%           2
------------------------------------------------------------
  Subtotal       Rs 460.00
  Tax            Rs 23.00
  Discount      -Rs 24.00
AMOUNT DUE: Rs 459.00
Success: Payment complete. Your cart is now empty.
```

Figure 5

5. CONCLUSION

The project implements a coherent small-store workflow connecting catalogue maintenance, cart operations, stock checks, tax and discount calculations, receipt presentation, and CSV transaction history. It demonstrates object-oriented decomposition, console interaction, input validation, and file handling in C++. Important limitations are the hard-coded administrator credentials, plain-text CSV storage without access control or backup, lack of concurrency and database transactions, absence of payment processing and refunds, and limited recovery if a file update fails partway through checkout. The current receipt is console-only, and there is no automated test suite or evidence of formal testing in the supplied materials. Future work could externalize and securely hash administrator credentials, introduce transactional database storage and backups, make inventory updates atomic, add automated unit/integration tests, improve audit and error recovery, support receipt export and reporting, and add a graphical interface only if the project scope requires it.

- **REFERENCES**

[1] main.cpp, inventory.h, customer.h, and bill.h, source files supplied with the Supermarket Billing and Inventory Management System mini project.

[2] inventory.csv and transactions.csv, sample data files supplied with the project.

[3] ISO/IEC 14882:2011, Programming Languages : C++, language and standard-library reference for the C++11-era constructs used by the implementation.

[5] Fnky/ANSI.md, ANSI escape codes and usage references. [Online]. Available: https://gist.github.com/fnky/458719343aabd01cfb17a3a4f7296797 Accessed: Oct. 22, 2026.